Лабораторная работа №3. Транспонирование матрицы и исследование локальности памяти

Вариант A: тип элемента double, $N_v = 3000$.

1. Характеристики системы

Процессор AMD Ryzen 7 5700U with Radeon Graphics: 8 ядер / 16 потоков, базовая частота 1,80 ГГц. Кеши (по Диспетчеру задач, суммарно): L1 512 КБ, L2 4 МБ, L3 8 МБ (на ядро: L1 64 КБ, L2 512 КБ; L3 общий). ОЗУ 15,3 ГБ, Windows 11. Компилятор: GCC ____ (MinGW-w64), сборка *gcc transpose.c -O2 -fopenmp*. Время измерялось omp_get_wtime(): прогрев, затем медиана из 5 запусков; так как таймер Windows грубый (около 1 мс), каждый замер усреднён по нескольким повторам подряд (повторов = $3 \cdot 10^7 / N^2 + 1$). Результат каждого запуска сверен с transpose_rows: несовпадений 0.

2. Результаты экспериментов

Таблица 1. Порядок обхода (1 поток)

N	Время rows, с	Время cols, с	BW rows, ГБ/с	BW cols, ГБ/с
256	0,000832	0,000303	1,26	3,46
512	0,005696	0,004383	0,74	0,96
1024	0,023000	0,017724	0,73	0,95
2048	0,099875	0,150625	0,67	0,45
3000	0,122250	0,116000	1,18	1,24

Таблица 2. Размер блока (1 поток, N = 3000)

Размер блока b	Время, с	BW, ГБ/с
4	0,045500	3,16
8	0,043000	3,35
16	0,041500	3,47
32	0,033250	4,33
64	0,029000	4,97
128	**0,028000**	**5,14**

Лучший размер блока b = 128.*

Таблица 3. Потоки (N = 3000, b = b* = 128)

Потоки	Время, с rows_omp	S rows_omp	Время, с blocked_omp	S blocked_omp
1	0,125000	0,98	0,026250	4,66
2	0,066500	1,84	0,020500	5,96
4	0,043000	2,84	0,016500	7,41
8	0,036250	3,37	0,014250	8,58
16	0,023000	5,32	0,014000	8,73

$S = T_1 / T_p$, T_1 = 0,12225 с (transpose_rows, 1 поток).

Таблица 4. False sharing (8 потоков, N = 3000, rows_omp)

Стратегия	Время, с	BW, ГБ/с
schedule(static)	0,036000	4,00
schedule(static, 1)	0,048250	2,98
schedule(static, 8)	0,037250	3,87

3. Вывод

В transpose_rows матрица A читается подряд, а B пишется с шагом 8N байт, в transpose_cols наоборот, поэтому в обеих версиях один из массивов обходится с шагом и каждое такое обращение тянет целую кеш-линию (64 Б) ради одного double. Разница заметна уже при N = 256–1024 (cols быстрее в 1,3–2,7 раза), при N = 2048 rows быстрее в 1,5 раза (шаг 16 КБ кратен степени двойки, вероятны конфликты кеша), а при N = 3000 версии почти равны (около 1,2 ГБ/с): каждая матрица занимает 72 МБ, что намного больше L3 (8 МБ).

Найденный b* = 128 лежит на границе проверенного диапазона: время падает с 0,0455 с (b = 4) до 0,028 с (b = 128), но от 64 к 128 выигрыш всего 3 %, то есть кривая выходит на плато. Пара блоков 128×128 занимает 2·128 КБ = 256 КБ: она не помещается в L1 (32 КБ данных на ядро), но помещается в L2 (512 КБ на ядро), а блоки $b \le 32$ лежат в L1, но проигрывают, вероятно, из-за накладных расходов циклов и неполного использования кеш-линий (при b = 4 строка блока 32 Б, меньше линии в 64 Б). Блочная версия на одном потоке быстрее transpose_rows в 4,4 раза (0,028 против 0,122 с).

Ускорение транспонирования ограничено сильнее, чем у интеграла из ЛР2: операция ничего не вычисляет и упирается в пропускную способность памяти, которую потоки делят между собой (общий L3 и каналы ОЗУ), тогда как вычисление интеграла ограничено ядрами и масштабируется почти линейно. Поэтому blocked_omp выходит на плато уже на 8 потоках (S = 8,58, при 16 потоках 8,73), а относительно собственного одного потока ускорение всего ×1,9 (0,0263 → 0,0140 с); rows_omp, у которого однопоточная версия плохо использует кеш, масштабируется дольше (S = 5,3 на 16 потоках), но всё равно медленнее блочной.

В эксперименте 4 schedule(static, 1) раздаёт соседние строки разным потокам, а элементы B[j·N+i] с соседними i лежат в одной кеш-линии (8 double = 64 Б), поэтому линия постоянно перемещается между ядрами (false sharing): 0,0483 с против 0,0360 с у schedule(static), где каждый поток пишет в свою непрерывную область (медленнее на 34 %). schedule(static, 8) близок к static (0,0373 с), так как порция из 8 строк почти совпадает с одной кеш-линией.